

Implementation of Dynamo in Go

Sadie Fisher

Dept. of Computer Science and Software Engineering

Cal Poly SLO

San Luis Obispo, USA

sfishe10@calpoly.edu

Abstract—As one of the largest e-commerce platforms in the world, Amazon.com depends on a system that is highly available and reliable. It must minimize or eliminate the risk of even small outages in order to maintain user trust and avoid negative financial consequences. In 2007, the storage system Dynamo was introduced as a solution to this requirement [1]. Some of the key features of Dynamo include high availability, extensive object versioning, and data replication. Though components of the system fail at times, the system's state and data are able to persist. This paper gives brief descriptions of the architecture of a Go implementation of Dynamo; it also points out differences between this implementation and that described in the original paper.

I. INTRODUCTION

In order to serve tens or hundreds of thousands of customers at once, Amazon's system requires data stores that are always available to be read from and written to. Dynamo achieves the needed availability and scalability by employing techniques such as consistent hashing, object versioning, and gossip-based protocol and failure detection. Though the Go implementation described in this paper only serves one client at a time, it exhibits the required durability of data, and its framework contains the major components needed to handle large amounts of data.

II. SYSTEM ARCHITECTURE

This implementation contains a server, 8 nodes, and a client; network I/O is achieved through Go's RPC package. The server handles requests from and responses to the client, as well as communication between nodes.

III. SYSTEM INTERFACE

The system exposes two operations to the client:

PUT <key> <value>

GET <key>

PUT inserts a key-value pair into the database, and GET returns a previously inserted key-value pair. Section VI will explain how different object versions are handled.

IV. PARTITIONING ALGORITHM

The 8 nodes are arranged in a "ring". For each one, a string describing its position (e.g. "node-1") is hashed to find the range of data values it is primarily responsible for in the ring. This node is considered the coordinator for values in that

range. When a key-value pair is inserted in a PUT operation, the key string is hashed (using the same hash function). The hash value determines where that key-value pair is stored in the ring.

V. REPLICATION

The system defines a value N , which is the number of nodes on which data is replicated. In this implementation, a default value of $N=3$ is used. This means that in addition to the coordinator node, which is in charge of the range in the ring containing the hashed value of the key from the PUT request, the next three healthy nodes following the coordinator node also store the data being inserted. These are the replica nodes.

VI. EXECUTION OF GET AND PUT OPERATIONS

Both PUT and GET requests are sent to the corresponding coordinator node first, which then contacts the replica nodes. Read and write quorums using thresholds (R and W where $R + W \leq N$) are used to confirm whether an operation was successful. This implementation uses default values of $W=R=2$.

For PUT operations, the coordinator updates the object's context, stores it locally, and then sends it to the replica nodes. It waits to receive W responses before notifying the client that the operation was successful.

Similarly, for GET operations, the coordinator contacts the replica nodes to collect all existing versions of the data. It waits to receive R responses before returning all collected versions, including its local version, to the client.

Because the database tracks context to support different object versions, the client keeps a map of contexts for each key it passes to the server. When a PUT operation is completed, the updated context is returned to the client to be stored in this map. Then the next time a PUT request is made for that key, the client fetches the stored context and sends it along with the request. For GET requests, since multiple versions of an object may exist, all are returned to the client.

VII. FAILURE DETECTION

This implementation uses gossip protocol to detect failures. Nodes frequently send their current membership table to their two "neighbors", and also update this table based on the tables received from other nodes. When they detect that another node has not increased its heartbeat in the last few seconds, that

Problem	Technique	Advantage
Partitioning	Consistent Hashing	Incremental Scalability
High availability for writes	Vector clocks with reconciliation during reads	Version size is decoupled from update rates
Handling temporary failures	Sloppy Quorum and Hinted Handoff	Provides high availability and durability
Membership and failure detection	Gossip-based membership protocol and failure detection	Preserves symmetry and avoids having a centralized registry for storing membership and node liveness information

TABLE I

node is marked as "dead" and removed from the table a few seconds later.

VIII. HANDLING FAILURES

Temporary failures are handled through hinted handoff.

During a PUT request, the coordinator waits a few seconds for responses from all of its replica nodes. If it does not receive an expected response, it marks that node as "dead" and instead sends the data to the next healthy node in the ring. This substitute node stores the data separately from its own data, keeping a "hint" in the metadata of which node was actually the intended recipient.

Similarly, during a GET request, the coordinator waits a few seconds for responses from its replica nodes. If it does not receive an expected response, it looks to the next healthy node in the ring to see if the data was stored there. That node checks its own hinted data, and if it contains an entry corresponding to the intended target and the queried key, it returns that entry to the coordinator.

During the regular membership table exchanges, if a substitute node notices that a dead node it was holding data for has returned, it transfers that data to the revived node. Once it receives confirmation of the transfer, it deletes the local data.

IX. DESIGN DECISIONS

Table I (taken from [1]) summarizes the techniques used by this implementation of Dynamo.

Due to time constraints, there are a couple features of Dynamo that are not present in this implementation:

- Permanent failure handling with Merkle Trees
- Virtual nodes for better distribution of data
- Support for multiple concurrent clients

REFERENCES

This system was implemented according to the design described in [1].

REFERENCES

- [1] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: amazon's highly available key-value store," *SIGOPS Oper. Syst. Rev.*, vol. 41, p. 205–220, Oct. 2007.